Coordenação em Sistemas Distribuídos

Algoritmo do Valentão (Bully)

Laboratório prático — Python 3 / sockets TCP

Entrega: implementação, experimentos reproduzíveis e análise.

1. Descrição do algoritmo

O Bully elege o processo de maior ID entre os participantes disponíveis. Ao detectar a ausência do coordenador, um processo envia ELECTION a todos os IDs superiores. Cada participante superior responde OK e inicia sua própria rodada. Quem recebeu OK deixa a decisão para um processo maior. Se nenhum superior responder, o processo se declara vencedor e anuncia COORDINATOR a todos os demais.

O resultado pressupõe comunicação eventual entre os sobreviventes e tempos de resposta compatíveis com os timeouts. A eleição não é um mecanismo de consenso tolerante a partições.

2. Arquitetura e protocolo

Cinco programas Python independentes, P1 a P5, executam no mesmo host. Os endereços são 127.0.0.1:5001 até 127.0.0.1:5005. Cada processo possui um servidor TCP, threads de atendimento, uma rodada de eleição por vez e, opcionalmente, um monitor de disponibilidade.

O protocolo usa JSON delimitado por newline, com campos tipo e origem. ELECTION recebe OK na mesma conexão. COORDINATOR recebe ACK. PING recebe PONG. Os sockets têm timeout de 0,5 segundo e limite de mensagem de 4096 bytes; o monitor opera a cada segundo. Após OK, espera-se até quatro segundos por um anúncio, antes de repetir a rodada.

O estado é protegido por lock. Uma geração local identifica anúncios recebidos durante uma eleição, evitando que uma rodada anterior sobrescreva o anúncio. O servidor já está escutando antes de iniciar o monitor. A interface oferece eleicao, status e sair. O modo --automatico habilita eleição inicial, detecção de falha e eleição na recuperação.

3. Evidências dos testes executados

O programa testar.py inicia subprocessos reais, envia comandos pela entrada padrão e consulta status até que todos os sobreviventes reconheçam o mesmo coordenador sem eleição em

andamento. As portas TCP padrão foram utilizadas. Os logs completos estão em evidencias/. A tabela abaixo apresenta os cenários verificados; os resultados seguintes foram capturados da execução.

Cenário	Resultado esperado
1. Cinco ativos, eleição em P5	P5
2. P5 encerra, eleição em P2	P4
3. P5 reinicia e solicita eleição	P5
4. P4 coordenador, P2 reinicia e solicita eleição	P4
5. Apenas P1/P2/P3, eleição em P1	P3
Eleições simultâneas em P1/P2/P3	P3
Modo automático: início, morte abrupta de P5, retorno de P5	P5 → P4 → P5

```
PASSOU: experimento 1, eleição inicial -> P5
PASSOU: experimento 2, falha de P5 -> P4
PASSOU: experimento 3, recuperação de P5 -> P5
PASSOU: experimento 4, recuperação de P2 mantém P4
PASSOU: experimento 5, apenas P1/P2/P3 -> P3
PASSOU: eleições simultâneas convergem para P3
PASSOU: inicialização, falha abrupta e recuperação automáticas
```

4. Logs e diagrama textual

As linhas abaixo são trechos reais dos logs. As setas mostram emissor, receptor e mensagem. O iniciador local, as desistências e o vencedor também aparecem. Cada processo registra sua própria perspectiva; não há coletor global nem relógio lógico. Os horários são os registrados pelo ambiente de execução.

manual-P2.log

```
[09:30:46.998328] P2 | aguardando conexões na porta 5002
Comandos: eleicao | status | sair
[P2] comando> [09:30:47.413565] P2 | P5 --> P2 : COORDINATOR (novo coordenador: P5)
[P2] comando> [09:30:48.072829] P2 | INICIANDO ELEIÇÃO (iniciador local: P2)
[09:30:48.073038] P2 | P2 --> P3 : ELECTION
[P2] comando> [09:30:48.079515] P2 | P3 --> P2 : OK; P2 desiste em favor de ID maior
[09:30:48.079606] P2 | P2 --> P4 : ELECTION
[09:30:48.081211] P2 | P4 --> P2 : OK; P2 desiste em favor de ID maior
[09:30:48.081293] P2 | P2 --> P5 : ELECTION
```

```
[09:30:48.081534] P2 | P5 indisponível / sem OK
[09:30:48.086989] P2 | P4 --> P2 : COORDINATOR (novo coordenador: P4)
[P2] comando> [09:30:48.885561] P2 | P5 --> P2 : COORDINATOR (novo coordenador: P5)
[P2] comando> [09:30:49.495684] P2 | P4 --> P2 : COORDINATOR (novo coordenador: P4)
[P2] comando> [09:30:50.246806] P2 | aguardando conexões na porta 5002
Comandos: eleicao | status | sair
[P2] comando> [09:30:50.461321] P2 | INICIANDO ELEIÇÃO (iniciador local: P2)
[09:30:50.461458] P2 | P2 --> P3 : ELECTION
[P2] comando> [09:30:50.466913] P2 | P3 --> P2 : OK; P2 desiste em favor de ID maior
[09:30:50.467058] P2 | P2 --> P4 : ELECTION
[09:30:50.470298] P2 | P4 --> P2 : OK; P2 desiste em favor de ID maior
[09:30:50.470476] P2 | P2 --> P5 : ELECTION
[09:30:50.470919] P2 | P5 indisponível / sem OK
[09:30:50.474604] P2 | P4 --> P2 : COORDINATOR (novo coordenador: P4)
[P2] comando> [09:30:51.133142] P2 | P1 --> P2 : ELECTION
[09:30:51.133624] P2 | P2 --> P1 : OK
[09:30:51.134300] P2 | INICIANDO ELEIÇÃO (iniciador local: P2)
[09:30:51.134522] P2 | P2 --> P3 : ELECTION
[09:30:51.138347] P2 | P3 --> P2 : OK; P2 desiste em favor de ID maior
[09:30:51.138479] P2 | P2 --> P4 : ELECTION
[09:30:51.138848] P2 | P4 indisponível / sem OK
[09:30:51.138948] P2 | P2 --> P5 : ELECTION
[09:30:51.139341] P2 | P5 indisponível / sem OK
[09:30:51.143583] P2 | P3 --> P2 : COORDINATOR (novo coordenador: P3)
[09:30:51.629203] P2 | P2 --> P3 : ELECTION
[09:30:51.630593] P2 | P1 --> P2 : ELECTION
[09:30:51.630862] P2 | P2 --> P1 : OK
[09:30:51.631819] P2 | P3 --> P2 : OK; P2 desiste em favor de ID maior
[09:30:51.631915] P2 | P2 --> P4 : ELECTION
[09:30:51.632259] P2 | P4 indisponível / sem OK
[09:30:51.632348] P2 | P2 --> P5 : ELECTION
```

manual-P4.log

```
[09:30:47.012500] P4 | aguardando conexões na porta 5004
Comandos: eleicao | status | sair
[P4] comando> [09:30:47.416715] P4 | P5 --> P4 : COORDINATOR (novo coordenador: P5)
[P4] comando> [09:30:48.080727] P4 | P2 --> P4 : ELECTION
[09:30:48.080951] P4 | P4 --> P2 : OK
[09:30:48.081277] P4 | INICIANDO ELEIÇÃO (iniciador local: P4)
[09:30:48.081392] P4 | P4 --> P5 : ELECTION
[09:30:48.084318] P4 | P5 indisponível / sem OK
[09:30:48.084436] P4 | VENCEDOR: P4 É O NOVO COORDENADOR
[09:30:48.084468] P4 | P4 --> P1 : COORDINATOR:4
[09:30:48.084678] P4 | P3 --> P4 : ELECTION
[09:30:48.084932] P4 | P4 --> P3 : OK
[09:30:48.085966] P4 | P4 --> P2 : COORDINATOR:4
[09:30:48.087578] P4 | P4 --> P3 : COORDINATOR:4
[09:30:48.088993] P4 | P4 --> P5 : COORDINATOR:4
[P4] comando> [09:30:48.888304] P4 | P5 --> P4 : COORDINATOR (novo coordenador: P5)
[P4] comando> [09:30:49.493184] P4 | INICIANDO ELEIÇÃO (iniciador local: P4)
[09:30:49.493305] P4 | P4 --> P5 : ELECTION
[P4] comando> [09:30:49.493595] P4 | P5 indisponível / sem OK
[09:30:49.493633] P4 | VENCEDOR: P4 É O NOVO COORDENADOR
[09:30:49.493650] P4 | P4 --> P1 : COORDINATOR:4
```

```
[09:30:49.494825] P4 | P4 --> P2 : COORDINATOR:4
[09:30:49.496144] P4 | P4 --> P3 : COORDINATOR:4
[09:30:49.497429] P4 | P4 --> P5 : COORDINATOR:4
[P4] comando> [09:30:50.469049] P4 | P3 --> P4 : ELECTION[09:30:50.469260] P4 | P2 --> P4 : ELECTION
[09:30:50.469762] P4 | P4 --> P3 : OK[09:30:50.469883] P4 | P4 --> P2 : OK
[09:30:50.470658] P4 | INICIANDO ELEIÇÃO (iniciador local: P4)
[09:30:50.471016] P4 | P4 --> P5 : ELECTION
[09:30:50.471322] P4 | P5 indisponível / sem OK
[09:30:50.471361] P4 | VENCEDOR: P4 É O NOVO COORDENADOR
[09:30:50.471381] P4 | P4 --> P1 : COORDINATOR:4
[09:30:50.473050] P4 | P4 --> P2 : COORDINATOR:4
[09:30:50.475859] P4 | P4 --> P3 : COORDINATOR:4
[09:30:50.478002] P4 | P4 --> P5 : COORDINATOR:4
```

automatico-P1.log

```
[09:30:52.268899] P1 | aguardando conexões na porta 5001
Comandos: eleicao | status | sair
[P1] comando> [09:30:53.255421] P1 | P5 --> P1 : COORDINATOR (novo coordenador: P5)
[P1] comando> [09:30:54.251846] P1 | P4 --> P1 : COORDINATOR (novo coordenador: P4)
[P1] comando> [09:30:55.730145] P1 | P5 --> P1 : COORDINATOR (novo coordenador: P5)
```

automatico-P5.log

```
[09:30:52.250114] P5 | aguardando conexões na porta 5005
Comandos: eleicao | status | sair
[P5] comando> [09:30:53.250207] P5 | Coordenador None indisponível; eleição automática
[09:30:53.250949] P5 | INICIANDO ELEIÇÃO (iniciador local: P5)
[09:30:53.251159] P5 | VENCEDOR: P5 É O NOVO COORDENADOR
[09:30:53.251201] P5 | P5 --> P1 : COORDINATOR:5
[09:30:53.255247] P5 | P4 --> P5 : ELECTION
[09:30:53.255542] P5 | P5 --> P4 : OK
[09:30:53.256067] P5 | P5 --> P2 : COORDINATOR:5
[09:30:53.257831] P5 | P5 --> P3 : COORDINATOR:5
[09:30:53.259634] P5 | P5 --> P4 : COORDINATOR:5
[09:30:53.266418] P5 | P2 --> P5 : PING; P5 --> P2 : PONG
[09:30:53.274091] P5 | P1 --> P5 : PING; P5 --> P1 : PONG[09:30:53.274251] P5 | P3 --> P5 : PING; P5
--> P3 : PONG
[P5] comando> [09:30:54.723694] P5 | aguardando conexões na porta 5005
Comandos: eleicao | status | sair
[P5] comando> [09:30:55.723864] P5 | Coordenador None indisponível; eleição automática
[09:30:55.724925] P5 | INICIANDO ELEIÇÃO (iniciador local: P5)
[09:30:55.725325] P5 | VENCEDOR: P5 É O NOVO COORDENADOR
[09:30:55.725421] P5 | P5 --> P1 : COORDINATOR:5
[09:30:55.730729] P5 | P5 --> P2 : COORDINATOR:5
[09:30:55.732457] P5 | P5 --> P3 : COORDINATOR:5
[09:30:55.733859] P5 | P5 --> P4 : COORDINATOR:5
```

5. Respostas às questões

1. Por que o maior ID tende a vencer? Cada participante consulta apenas IDs maiores. Enquanto houver um superior disponível respondendo OK, o inferior não vence. O maior

disponível não recebe OK de nenhum superior e anuncia sua coordenação.

2. P1 a P6, P6 falha e P2 inicia: quem recebe ELECTION? P2 envia para P3, P4, P5 e P6. P3, P4 e P5 recebem e respondem; P6 é destino de uma tentativa, mas, estando indisponível, não recebe a mensagem nem responde. Nas rodadas subsequentes, P3 consulta P4/P5/P6, P4 consulta P5/P6 e P5 consulta P6. P5 vence.

3. P5 falha e P2 inicia: sequência esperada. P2 envia ELECTION para P3/P4/P5; P3/P4 respondem OK. P3 consulta P4/P5, e P4 responde OK. P4 consulta P5 e não recebe resposta. P2 e P3 desistem em favor de IDs superiores; P4 vence e envia COORDINATOR:4 aos demais disponíveis. A ordem exata pode variar, pois as threads executam concorrentemente.

4. P4 coordenador e P5 volta. Se P5 iniciar uma eleição, não terá IDs superiores entre os cinco configurados. Assim, vence e anuncia COORDINATOR:5. No modo manual o usuário dispara a eleição; no automático o monitor a inicia ao encontrar o coordenador local indefinido.

5. Por que muitas mensagens? Uma eleição iniciada por um ID baixo pode desencadear eleições em todos os superiores. A soma das consultas pode atingir $n(n-1)/2$, além das respostas OK, anúncios e repetições. Portanto, o custo pode ser quadrático em n. Monitores e eleições simultâneas acrescentam tráfego.

6. Limitações e discussão

A configuração é fixa, os IDs precisam ser únicos e todos os participantes precisam conhecer os pares. Falhas são inferidas por timeout: atraso de rede, sobrecarga e pausas de execução podem provocar suspeitas falsas. Durante partições, diferentes grupos podem eleger coordenadores próprios. Não há quórum, autenticação, persistência ou fencing para impedir ações de um coordenador antigo. Mensagens tardias podem causar novas rodadas; a implementação busca convergência no laboratório local, sem garantir uma ordem global de eventos.

PING/PONG testa se um processo responde, sem provar que o sistema inteiro concorda com ele. Os testes demonstram os cenários especificados na máquina local, não constituem prova formal para todas as intercalações, falhas ou redes possíveis. Uma alternativa para produção exigiria mecanismos de consenso e controle de autoridade apropriados.

7. Reprodução e conclusão

Execute python3 testar.py com as cinco portas livres. Para experimentar manualmente, abra cinco terminais e execute python3 processo.py ID em cada um. O README detalha os comandos. Todos os cenários executados convergiram para o maior ID disponível. O retorno de um participante superior permite que ele recupere a coordenação após iniciar uma eleição.

Apêndice — Código desenvolvido

config.py

```
"""Endereços TCP e tempos (em segundos) do laboratório."""
PROCESSOS = {pid: ("127.0.0.1", 5000 + pid) for pid in range(1, 6)}
TIMEOUT = 0.5
INTERVALO_PING = 1.0
ESPERA_COORDENADOR = 4.0
```

processo.py

```
"""Algoritmo Bully com processos independentes e mensagens TCP delimitadas por newline."""
import argparse
import json
import socket
import threading
import time
from datetime import datetime

from config import PROCESSOS, TIMEOUT, INTERVALO_PING, ESPERA_COORDENADOR


def ler_mensagem(conexao):
    dados = bytearray()
    while len(dados) < 4096:
        parte = conexao.recv(1)
        if not parte:
            raise ValueError("Conexão encerrada antes do fim da mensagem")
        if parte == b"\n":
            mensagem = json.loads(dados)
            if not isinstance(mensagem, dict):
                raise ValueError("Mensagem deve ser um objeto")
            return mensagem
        dados.extend(parte)
    raise ValueError("Mensagem excede 4096 bytes")


def escrever(conexao, mensagem):
    conexao.sendall((json.dumps(mensagem) + "\n").encode())


class Processo:
    def __init__(self, id_processo, automatico=False, enderecos=None):
        self.processos = enderecos if enderecos is not None else PROCESSOS
        if id_processo not in self.processos:
            raise ValueError("ID não configurado")
        self.id = id_processo
        self.coordenador = None
        self.automatico = automatico
        self.eleicao_em_andamento = False
        self.lock = threading.RLock()
        self.parado = threading.Event()
```

```
    self.anuncio = threading.Event()
    self.geracao = 0
    self.servidor = None
    self.threads = []

def log(self, texto):
    print(f"[{datetime.now():%H:%M:%S.%f}] P{self.id} | {texto}", flush=True)

def iniciar(self):
    servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    try:
        servidor.bind(self.processos[self.id])
        servidor.listen()
        servidor.settimeout(0.2)
    except OSError:
        servidor.close()
        raise
    self.servidor = servidor
    self._thread(self.iniciar_servidor)
    if self.automatico:
        self._thread(self.monitorar)
    self.log(f"aguardando conexões na porta {self.processos[self.id][1]}")

def _thread(self, alvo, *args):
    thread = threading.Thread(target=alvo, args=args, daemon=True)
    self.threads.append(thread)
    thread.start()

def iniciar_servidor(self):
    while not self.parado.is_set():
        try:
            conexao, _ = self.servidor.accept()
        except socket.timeout:
            continue
        except OSError:
            break
        self._thread(self.atender, conexao)

def atender(self, conexao):
    with conexao:
        conexao.settimeout(TIMEOUT)
        try:
            msg = ler_mensagem(conexao)
            origem, tipo = msg.get("origem"), msg.get("tipo")
            if type(origem) is not int or origem not in self.processos:
                raise ValueError("Origem inválida")
            if tipo == "ELECTION" and origem < self.id:
                self.log(f"P{origem} --> P{self.id} : ELECTION")
                escrever(conexao, {"tipo": "OK", "origem": self.id})
                self.log(f"P{self.id} --> P{origem} : OK")
                self.solicitar_eleicao()
            elif tipo == "COORDINATOR":
                if origem < self.id:
                    self.log(f"Anúncio de P{origem} inferior; iniciando eleição")
                    self.solicitar_eleicao()
```

```
                    else:
                        with self.lock:
                            self.coordenador = origem
                            self.geracao += 1
                            self.anuncio.set()
                        self.log(f"P{origem} --> P{self.id} : COORDINATOR (novo coordenador:
P{origem})")
                    escrever(conexao, {"tipo": "ACK", "origem": self.id})
                elif tipo == "PING":
                    self.log(f"P{origem} --> P{self.id} : PING; P{self.id} --> P{origem} : PONG")
                    escrever(conexao, {"tipo": "PONG", "origem": self.id})
                else:
                    raise ValueError("Tipo ou direção inválida")
            except (OSError, ValueError, TypeError) as erro:
                self.log(f"Conexão ignorada: {erro}")

    def enviar(self, destino, tipo):
        try:
            with socket.create_connection(self.processos[destino], TIMEOUT) as cliente:
                cliente.settimeout(TIMEOUT)
                escrever(cliente, {"tipo": tipo, "origem": self.id})
                resposta = ler_mensagem(cliente)
                if resposta.get("origem") == destino:
                    return resposta.get("tipo")
        except (OSError, ValueError, TypeError):
            return None
        return None

    def solicitar_eleicao(self):
        with self.lock:
            if self.eleicao_em_andamento or self.parado.is_set():
                return
            self.eleicao_em_andamento = True
        self._thread(self.iniciar_eleicao)

    def iniciar_eleicao(self):
        try:
            while not self.parado.is_set():
                with self.lock:
                    geracao = self.geracao
                    self.anuncio.clear()
                self.log(f"INICIANDO ELEIÇÃO (iniciador local: P{self.id})")
                respondeu = False
                for pid in sorted(self.processos):
                    if pid > self.id:
                        self.log(f"P{self.id} --> P{pid} : ELECTION")
                        if self.enviar(pid, "ELECTION") == "OK":
                            respondeu = True
                            self.log(f"P{pid} --> P{self.id} : OK; P{self.id} desiste em favor de ID
maior")
                        else:
                            self.log(f"P{pid} indisponível / sem OK")
                if self.parado.is_set():
                    return
                with self.lock:
                    if self.geracao != geracao:
```

```
                        return
                    if not respondeu:
                        self.coordenador = self.id
                        self.geracao += 1
                        self.log(f"VENCEDOR: P{self.id} É O NOVO COORDENADOR")
                if not respondeu:
                    for pid in sorted(self.processos):
                        if pid != self.id:
                            self.log(f"P{self.id} --> P{pid} : COORDINATOR:{self.id}")
                            self.enviar(pid, "COORDINATOR")
                    return
                # Se o processo maior morrer após OK, não esperamos indefinidamente.
                if self.anuncio.wait(ESPERA_COORDENADOR):
                    return
                self.log("Prazo para COORDINATOR expirou; repetindo eleição")
        finally:
            with self.lock:
                self.eleicao_em_andamento = False

    def monitorar(self):
        while not self.parado.wait(INTERVALO_PING):
            with self.lock:
                coordenador = self.coordenador
                ocupado = self.eleicao_em_andamento
            if ocupado or coordenador == self.id:
                continue
            if coordenador is None or self.enviar(coordenador, "PING") != "PONG":
                self.log(f"Coordenador {coordenador} indisponível; eleição automática")
                self.solicitar_eleicao()

    def encerrar(self):
        self.parado.set()
        self.anuncio.set()
        if self.servidor:
            self.servidor.close()
        for thread in list(self.threads):
            if thread is not threading.current_thread():
                thread.join(timeout=TIMEOUT + 0.3)


def main():
    parser = argparse.ArgumentParser(description="Laboratório do algoritmo Bully")
    parser.add_argument("id", type=int, choices=sorted(PROCESSOS))
    parser.add_argument("--automatico", action="store_true", help="ativa PING/PONG e eleição ao
iniciar")
    args = parser.parse_args()
    processo = Processo(args.id, args.automatico)
    try:
        processo.iniciar()
        print("Comandos: eleicao | status | sair", flush=True)
        while True:
            comando = input(f"[P{args.id}] comando> ").strip().lower()
            if comando == "eleicao":
                processo.solicitar_eleicao()
            elif comando == "status":
                with processo.lock:
```

```
                    coord = f"P{processo.coordenador}" if processo.coordenador else "indefinido"
                    print(f"ID: P{processo.id} | Coordenador: {coord} | Eleição:
{processo.eleicao_em_andamento}", flush=True)
            elif comando == "sair":
                break
            else:
                print("Comandos: eleicao | status | sair", flush=True)
    except (EOFError, KeyboardInterrupt):
        pass
    except OSError as erro:
        parser.exit(1, f"Não foi possível iniciar P{args.id}: {erro}\n")
    finally:
        processo.encerrar()


if __name__ == "__main__":
    main()
```

testar.py

```
"""Testes de integração com cinco subprocessos reais e sockets TCP locais."""
import subprocess
import sys
import time
from pathlib import Path

RAIZ = Path(__file__).resolve().parent


def executar(automatico=False):
    processos, arquivos = {}, []
    sufixo = "automatico" if automatico else "manual"

    def iniciar(pid):
        arquivo = (RAIZ / "evidencias" / f"{sufixo}-P{pid}.log").open("a")
        arquivos.append(arquivo)
        processos[pid] = subprocess.Popen(
            [sys.executable, "-u", str(RAIZ / "processo.py"), str(pid)]
            + (["--automatico"] if automatico else []),
            stdin=subprocess.PIPE, stdout=arquivo, stderr=subprocess.STDOUT, text=True)

    def comando(pid, texto):
        processos[pid].stdin.write(texto + "\n")
        processos[pid].stdin.flush()

    def parar(pid, abrupto=False):
        processo = processos.pop(pid)
        if abrupto:
            processo.kill()
        else:
            processo.stdin.write("sair\n")
            processo.stdin.flush()
        processo.wait(timeout=10)
        try:
            processo.stdin.close()
```

```
        except BrokenPipeError:
            pass

    def aguardar(vencedor):
        limite = time.monotonic() + 15
        while time.monotonic() < limite:
            offsets = {pid: (RAIZ / "evidencias" / f"{sufixo}-P{pid}.log").stat().st_size for pid in
processos}
            for pid in processos:
                assert processos[pid].poll() is None, f"P{pid} encerrou inesperadamente"
                comando(pid, "status")
            time.sleep(0.25)
            if all(f"Coordenador: P{vencedor} | Eleição: False" in
                   (RAIZ / "evidencias" / f"{sufixo}-P{pid}.log").read_bytes()
[offsets[pid]:].decode()
                   for pid in processos):
                return
        raise AssertionError(f"Não houve convergência para P{vencedor}")

    try:
        for pid in range(1, 6):
            iniciar(pid)
        time.sleep(0.5)
        if automatico:
            aguardar(5)
            parar(5, abrupto=True)
            aguardar(4)
            iniciar(5)
            aguardar(5)
            print("PASSOU: inicialização, falha abrupta e recuperação automáticas")
        else:
            comando(5, "eleicao")
            aguardar(5)
            print("PASSOU: experimento 1, eleição inicial -> P5")
            parar(5)
            comando(2, "eleicao")
            aguardar(4)
            print("PASSOU: experimento 2, falha de P5 -> P4")
            iniciar(5)
            time.sleep(0.3)
            comando(5, "eleicao")
            aguardar(5)
            print("PASSOU: experimento 3, recuperação de P5 -> P5")
            parar(5)
            comando(4, "eleicao")
            aguardar(4)
            parar(2)
            iniciar(2)
            time.sleep(0.3)
            comando(2, "eleicao")
            aguardar(4)
            print("PASSOU: experimento 4, recuperação de P2 mantém P4")
            parar(4)
            comando(1, "eleicao")
            aguardar(3)
            print("PASSOU: experimento 5, apenas P1/P2/P3 -> P3")
```

```
            for pid in processos:
                comando(pid, "eleicao")
            aguardar(3)
            print("PASSOU: eleições simultâneas convergem para P3")
    finally:
        for pid in list(processos):
            parar(pid, abrupto=True)
        for arquivo in arquivos:
            arquivo.close()


if __name__ == "__main__":
    (RAIZ / "evidencias").mkdir(exist_ok=True)
    for caminho in (RAIZ / "evidencias").glob("*.log"):
        caminho.unlink()
    executar()
    executar(automatico=True)
```